

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico Parte 3



8 de febrero de 2025

Nombre	Padrón
Maximo Giovanettoni	110303
Franco Bossi	111122
Luca Saluzzi	108088
Agustin Conti	107800
Gabriel Peralta	101767

1. Introducción

En la parte 3 del Trabajo Práctico se aborda la resolución de un juego de mesa de un único jugador conocido como la Batalla Naval Individual. En este juego tenemos un tablero de $n \times m$ casilleros, y k barcos. Cada barco tiene un largo, el cuál requiere la misma cantidad de casilleros para ser ubicado. Todos los barcos tienen 1 casillero de ancho. El tablero a su vez tiene un requisito de consumo tanto en sus filas como en sus columnas.

En el siguiente informe se dan demostraciones de que el problema está en np, y de que además es np-completo. También se detalla una resolución usando la técnica de backtracking. Por último se muestra un algoritmo de aproximación propuesto por el almirante John Jellicoe.

Ejemplo de Resolución

2. Demostraciones de NP y NP-Completo

2.1. El problema de la batalla naval está en NP

Para demostrar que el problema de la batalla naval está en NP se debe proponer un validador que verifique, en tiempo polinomial, que un arreglo de barcos es solución del problema.

Se propone el siguiente validador:

Se requieren las listas de demandas por filas y por columnas, y las posiciones de todos los barcos de una solución dada. Por cada barco de la solución, se verifica que este en una posición vertical u horizontal y que pueda ser ingresado en el tablero en esa posición cumpliendo todas las restricciones del juego, como por ejemplo, que no esté adyacente a otro barco. Así, todos los barcos deben entrar en el tablero o no será una solución válida.

```
1 # Solucion es una lista de tuplas (fila_ini, columna_ini, fila_fin, columna_fin)
  con las posiciones de cada barco
2 def validador_naual(demandas_filas, demandas_columnas, solucion):
3     n = len(demandas_filas)
4     m = len(demandas_columnas)
5     acumulados_fila = [0] * len(demandas_filas)
6     acumulados_columna = [0] * len(demandas_columnas)
7     tablero = [[0] * m for _ in range(n)]
8     for barco in solucion: # 0(k)
9         if (barco[0] != barco[2] and barco[1] != barco[3]) or (barco[0] == barco[2]
10             and barco[3] < barco[1] or
11                 barco[1] == barco[3]
12                 and barco[2] < barco[0]):
13             return False
14         # 0(n x m)
15         if barco[0] == barco[2] and not ubicar_barco_horizontal(barco[3] - barco[1]
16             + 1, tablero, demandas_filas,
17                 demandas_columnas,
18                 acumulados_fila, acumulados_columna):
19             return False
20         # 0(n x m)
21         if barco[1] == barco[3] and not ubicar_barco_vertical(barco[2] - barco[0] +
22             1, tablero, demandas_filas,
23                 demandas_columnas,
24                 acumulados_fila, acumulados_columna):
25             return False
26     return True
```

Se muestra como se ubican los barcos horizontalmente. La ubicacion de forma vertical es análoga.

```
1 # Verifica que el barco a ubicar no sea adyacente a otro ya ubicado
2 def adyacente_horizontal(barco, tablero, fila, pos_ini, n, m):
3     if pos_ini > 0 and (tablero[fila][pos_ini - 1] == 1 or (fila > 0 and tablero[
4         fila - 1][pos_ini - 1] == 1) or
5         (fila < n - 1 and tablero[fila + 1][pos_ini - 1] == 1)):
6         return False
7     if pos_ini + barco < m and (tablero[fila][pos_ini + barco] == 1 or
8         (fila > 0 and tablero[fila - 1][pos_ini + 1] == 1)
9         or
10         (fila < n - 1 and tablero[fila + 1][pos_ini + 1] ==
11         1)):
12         return False
13     for i in range(pos_ini, pos_ini + barco): # 0(L) con L el largo del barco
14         actual
15         if (fila > 0 and tablero[fila - 1][i] == 1) or (fila < n - 1 and tablero[
16             fila + 1][i] == 1):
```

```
13         return False
14
15     return True

1  # Ubica un barco de forma horizontal en el tablero
2  def ubicar_barco_horizontal(barco, tablero, demandas_filas, demandas_columnas,
3      acum_fila, acum_columna):
4      n = len(demandas_filas)
5      m = len(demandas_columnas)
6      for i in range(n): # O(m x n)
7          if acum_fila[i] + barco > demandas_filas[i]:
8              continue
9
10         pos_primer_cero = -1
11         for j in range(m): # O(m)
12             if tablero[i][j] == 1 or (acum_columna[j] + 1 > demandas_columnas[j]):
13                 pos_primer_cero = -1
14                 continue
15             if pos_primer_cero == -1:
16                 pos_primer_cero = j
17             if j - pos_primer_cero == barco - 1:
18                 if not adyacente_horizontal(barco, tablero, i, pos_primer_cero, n,
19                     m):
20                     pos_primer_cero = -1
21                     continue
22                 asignar_barco_horizontal(barco, tablero, i, pos_primer_cero,
23                     acum_columna)
24                 acum_fila[i] += barco
25                 return True
26
27     return False
```

La complejidad del algoritmo propuesto es $\mathcal{O}(k * n * m)$, con k la cantidad de barcos en la solución. Por lo tanto el problema está en NP.

2.2. El problema de la batalla naval es NP-Completo

Para demostrar que el problema de la batalla naval es np-completo se debe poder reducir bin packing (que es np-completo) al problema de la batalla naval.

Bin packing - problema de decisión: dado un arreglo de números y B bins, cada uno de capacidad C , con la suma de los números igual a $B * C$; se debe poder decidir si los números pueden dividirse en B bins de tal forma que la sumatoria de los números en cada bin sea igual a C .

Reducción propuesta: Se puede formar 1 tablero de $(2BC) \times B$ ($2BC$ filas, B columnas), la demanda de cada fila será de 1, y la demanda de cada columna será de C . La demanda total sería de $3BC$. Con esto se puede llamar al problema de decisión de la batalla naval, con el arreglo de números visto como barcos, y verificar que la demanda total cumplida sea de $2 * B * C$.

La idea es que, con los números vistos como barcos, cada barco sólo se pueda colocar de forma vertical y cumpla con la demanda C de cada columna.

Entonces existe una solución de Bin Packing para el arreglo de números y B bins de capacidad C si y solo si la demanda total cumplida para el tablero equivalente y el arreglo de numeros visto como barcos en el problema de la batalla naval es de $2 * B * C$.

Demostración:

- Si con el arreglo de números se puede llenar B bins de capacidad C entonces la sumatoria total de los numeros es de $B * C$. Si vemos los números como un arreglo de barcos, la demanda que se podrá satisfacer es de $2 * B * C$ si entran todos los barcos. Con esto, solo queda ver si entran todos los barcos; como la demanda de cada fila es 1, los barcos solo pueden entrar de forma vertical y sin solaparse entre las filas. Tenemos $2 * B * C$ filas por lo que todos los barcos pueden entrar en una sola fila respetando la unidad de espacio entre cada barco. Se tienen B columnas, si se excede la demanda C de cada columna se puede ubicar en la columna siguiente. Así, la demanda total cumplida será de $2 * B * C$.

- Si la demanda total cumplida es de $2 * B * C$ quiere decir que se tuvo que satisfacer la demanda C de cada columna en las B columnas, como los barcos solo se pueden ubicar en vertical inmediatamente se cumple que la suma del largo de los barcos en las B columnas es igual C .

Esta transformación es polinomial. Por lo tanto el problema de la batalla naval es NP-Completo

3. Explicación del algoritmo de Backtracking

Nosotros hemos decidido en este caso en no recorrer la matriz completa sino en antes poner en dos diccionarios (de filas y columnas) de los lugares posibles los cuales el barco seleccionado podría entrar en base a su valor siendo que no se salga del tablero y que no sea mayor a la demanda de la fila o columna. Luego, en base a estos diccionarios, se va a ir probando todas las combinaciones posibles de barcos en el tablero y se va a ir guardando la mejor solución encontrada.

Se hizo en esta funcion:

```
1 def buscar_pos_disponibles_para_barcos(i, j, columnas, filas,
2     dicc_posiciones_columnas, dicc_posiciones_filas, largo, max_barco):
3     if filas[i] == 0 or columnas[j] == 0:
4         return dicc_posiciones_columnas, dicc_posiciones_filas
5     columnas_cumple = False
6     filas_cumple = False
7     if filas[i] > 0:
8         dicc_posiciones_filas[1].add((i,j))
9         filas_cumple = True
10    if columnas[j] > 0:
11        dicc_posiciones_columnas[1].add((i,j))
12        columnas_cumple = True
13    for largo in range(2, max_barco+1):
14        if not filas_cumple and not columnas_cumple:
15            break
16        if columnas_cumple and (i, j-1) in dicc_posiciones_filas[largo-1] and filas
17        [i] >= largo:
18            dicc_posiciones_filas[largo].add((i,j))
19        else:
20            columnas_cumple = False
21            if filas_cumple and (i-1, j) in dicc_posiciones_columnas[largo-1] and
22            columnas[j] >= largo:
23                dicc_posiciones_columnas[largo].add((i,j))
24            else:
25                filas_cumple = False
26    return dicc_posiciones_columnas, dicc_posiciones_filas
```

Ademas, se han considerando unas podas para que el algoritmo sea mas eficiente y para que no se recorran todas las posibles combinaciones de barcos en el tablero. Siendo esto una de las bases del Backtracking Estas podas son las siguientes:

3.1. Primera poda

Si el indice a recorrer ya recorrio todos los barcos o si la suma de las demandas de las filas y columnas es 0 o si la demanda de la fila o columna es 0 o si el barco que se quiere poner en el tablero es mayor a la demanda de la fila o columna, entonces se retorna la mejor solución encontrada hasta el momento.

```
1 if (idx >= len(barcos) or suma_columnas == 0 or suma_filas == 0 or
2     barcos[-1][0] > suma_columnas or barcos[-1][0] > suma_filas):
3
4     if suma_columnas + suma_filas < mejor_solucion[1]:
5         mejor_solucion[0] = copy.deepcopy(matriz)
6         mejor_solucion[1] = suma_columnas + suma_filas
7         mejor_solucion[2] = copy.deepcopy(posiciones_barcos)
8     return mejor_solucion
```

3.2. Segunda poda

Si el estado actual ya fue visitado, entonces se retorna esa solución

```
1 if estado_actual in visitados:
2     return visitados[estado_actual]
```

3.3. Tercera poda

Si no hay posiciones disponibles para el barco que se quiere poner en el tablero, se avanza al siguiente barco.

```
1 if not posiciones_por_fila[barco] and not posiciones_por_columna[barco]:
2     return backtrack(
3         matriz, barcos, idx + 1, demanda_filas, demanda_columnas,
4         suma_filas, suma_columnas, mejor_solucion, posiciones_por_fila,
5         posiciones_por_columna, demanda_restante - barco,
6         visitados, posiciones_barcos
7     )
```

3.4. Cuarta poda

Si la suma de las demandas de las filas y columnas menos el doble de la demanda restante es mayor o igual a la mejor solución encontrada hasta el momento, se corta la rama.

```
1 if (suma_columnas + suma_filas) - (2 * demanda_restante) >= mejor_solucion[1]:
2     return mejor_solucion
```

3.5. Quinta poda

Si cumple con toda la demanda, se retorna esa solución.

```
1 if mejor_solucion[1] == 0:
2     return mejor_solucion
```

4. Algoritmo de John Jellicoe

El almirante J.J. nos propone un algoritmo de aproximación para El Problema de la Batalla Naval. Buscamos la fila/columna con mayor demanda y ubicamos el barco de mayor longitud ahí. De no poder ubicarlo, lo saltamos y probamos con el siguiente hasta que no queden más barcos o no haya más demandas a cumplir.

4.1. Algoritmo completo

```
1 def algoritmo_JJ(n, m, largo_barcos, filas_restricciones, col_restricciones):
2     tablero = np.zeros((n, m), dtype=int)
3
4     barcos = sorted(largo_barcos, reverse=True) # O(k log k)
5
6     while barcos:
7         filas_demandas = [(i, filas_restricciones[i]) for i in range(n) if
8             filas_restricciones[i] > 0] # O(n+m)
9         col_demandas = [(j, col_restricciones[j]) for j in range(m) if
10             col_restricciones[j] > 0] # O(n+m)
11
12         if not filas_demandas and not col_demandas:
13             break
```

```

13     barco_colocado = False # para verificar si el barco se coloc en esta
iteración
14
15     for barco in barcos:
16         # Intentar colocar el barco en una fila
17         if filas_demandas and (not col_demandas or max(filas_demandas, key=
lambda x: x[1])[1] >= max(col_demandas, key=lambda x: x[1])[1]):
18             fila_i = max(filas_demandas, key=lambda x: x[1])[0]
19             if filas_restricciones[fila_i] >= barco: # Verificar si la fila
puede acomodar el barco
20                 for columna_inicio in range(m):
21                     if all(0 <= columna_inicio + i < m and col_restricciones[
columna_inicio + i] > 0 for i in range(barco)) and puede_poner_barco(tablero, n
, m, fila_i, columna_inicio, barco, True):
22                         colocar_barco(tablero, fila_i, columna_inicio, barco,
True)
23                         filas_restricciones[fila_i] -= barco
24                         for i in range(barco):
25                             col_restricciones[columna_inicio + i] -= 1
26                         barcos.remove(barco)
27                         barco_colocado = True
28                         break
29             if barco_colocado:
30                 break
31
32         # Intentar colocar el barco en una columna
33         if col_demandas and not barco_colocado:
34             columna_i = max(col_demandas, key=lambda x: x[1])[0]
35             if col_restricciones[columna_i] >= barco: # Verificar si la
columna puede acomodar el barco
36                 for fila_inicio in range(n):
37                     if all(0 <= fila_inicio + i < n and filas_restricciones[
fila_inicio + i] > 0 for i in range(barco)) and puede_poner_barco(tablero, n, m
, fila_inicio, columna_i, barco, False):
38                         colocar_barco(tablero, fila_inicio, columna_i, barco,
False)
39                         col_restricciones[columna_i] -= barco
40                         for i in range(barco):
41                             filas_restricciones[fila_inicio + i] -= 1
42                         barcos.remove(barco)
43                         barco_colocado = True
44                         break
45             if barco_colocado:
46                 break
47
48         # Si no se pudo colocar ning n barco en esta iteración, salimos del bucle
49         if not barco_colocado:
50             break
51
52     demanda_restante = sum(filas_restricciones) + sum(col_restricciones)
53     return tablero, demanda_restante
54
55

```

4.2. Complejidad del Algoritmo JJ

Primero ordenamos la lista de barcos, lo que es $O(k \log k)$ siendo k la cantidad de barcos.

```
1 barcos = sorted(largo_barcos, reverse=True)
```

Extraer `filas_demandas` y `col_demandas` es $O(n + m)$.

```

1 filas_demandas = [(i, filas_restricciones[i]) for i in range(n) if
filas_restricciones[i] > 0]
2 col_demandas = [(j, col_restricciones[j]) for j in range(m) if col_restricciones[j]
> 0]

```

La función `puede_poner_barco` itera sobre el largo l de un barco $O(l)$, y verifica los adyacentes en cada punto $O(l \cdot 8)$. Por lo que la complejidad de `puede_poner_barco` es $O(l)$.

```
1 def puede_poner_barco(tablero, n, m, x, y, largo, es_horizontal):
2     if es_horizontal:
3         if y + largo > m:
4             return False
5         for i in range(largo): # revisar adyacentes si es horizontal
6             if tablero[x, y + i] == 1 or \
7                 (x > 0 and tablero[x - 1, y + i] == 1) or \
8                 (x < n - 1 and tablero[x + 1, y + i] == 1) or \
9                 (x > 0 and y + i > 0 and tablero[x - 1, y + i - 1] == 1) or \
10                (x > 0 and y + i < m - 1 and tablero[x - 1, y + i + 1] == 1) or \
11                (x < n - 1 and y + i > 0 and tablero[x + 1, y + i - 1] == 1) or \
12                (x < n - 1 and y + i < m - 1 and tablero[x + 1, y + i + 1] == 1):
13             return False
14         if y > 0 and tablero[x, y - 1] == 1:
15             return False
16         if y + largo < m and tablero[x, y + largo] == 1:
17             return False
18     else:
19         if x + largo > n:
20             return False
21         for i in range(largo): # revisar adyacentes si no es horizontal
22             if tablero[x + i, y] == 1 or \
23                 (y > 0 and tablero[x + i, y - 1] == 1) or \
24                 (y < m - 1 and tablero[x + i, y + 1] == 1) or \
25                 (x + i > 0 and y > 0 and tablero[x + i - 1, y - 1] == 1) or \
26                 (x + i > 0 and y < m - 1 and tablero[x + i - 1, y + 1] == 1) or \
27                 (x + i < n - 1 and y > 0 and tablero[x + i + 1, y - 1] == 1) or \
28                 (x + i < n - 1 and y < m - 1 and tablero[x + i + 1, y + 1] == 1):
29             return False
30         if x > 0 and tablero[x - 1, y] == 1:
31             return False
32         if x + largo < n and tablero[x + largo, y] == 1:
33             return False
34     return True
```

La función colocar_barco es $O(l)$ ya que itera sobre el largo del barco.

```
1 def colocar_barco(tablero, x, y, largo, es_horizontal):
2     if es_horizontal:
3         for i in range(largo):
4             tablero[x, y + i] = 1
5     else:
6         for i in range(largo):
7             tablero[x + i, y] = 1
```

El peor caso para un barco individual sería verificar todas las posiciones del tablero $O(n \cdot m)$ y el barco $O(l)$, lo que sería $O(n \cdot m \cdot l)$. La complejidad total del peor caso del ciclo while sería $O(n \cdot m \cdot L)$. Con $L = \sum l_i$.

Sumando todos los componentes juntos:

- **Ordenamiento de los barcos:** $O(k \log k)$.
- **Colocación de barcos:** $O(n \cdot m \cdot L)$.
- **Procesamiento de fil.demandas y col.demandas en cada iteración:** $O(k \cdot (n + m))$.

El término dominante es el paso de colocación de barcos, por lo que la complejidad general del algoritmo es:

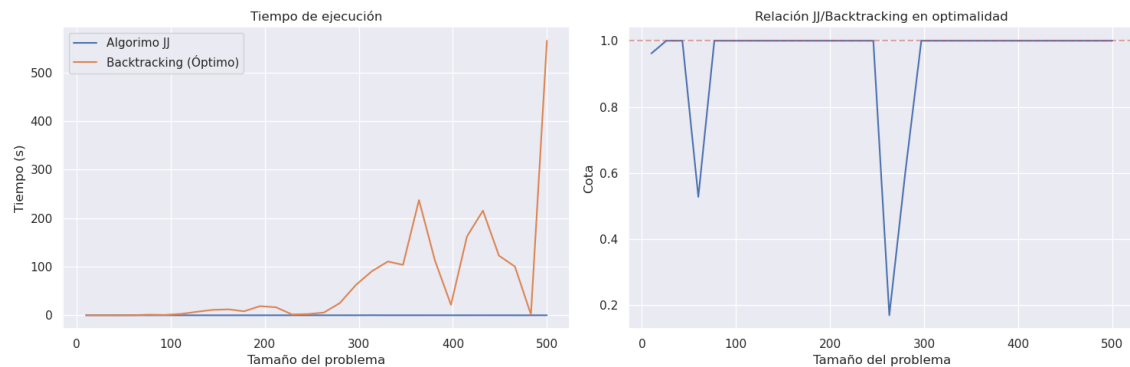
$$O(n \cdot m \cdot L)$$

donde:

- n : Número de filas.
- m : Número de columnas.
- L : Longitud total de todos los barcos.

5. Optimalidad del Algoritmo

Ya que este algoritmo es una aproximación, nos gustaría saber que tan bueno es, que tanto se acerca al resultado óptimo que se podría obtener con algoritmos mas costosos (como por ejemplo, backtracking). Para esto creamos una función de casos de prueba aleatorios que genera diferentes tableros con demandas y largos de barcos randomizados. Luego, mediante una Jupyter Notebook corrimos varias iteraciones para generar entonces el siguiente gráfico.



Éste gráfico se generó a partir de 50 iteraciones de tablero de $n \times n$ con n entre 10 y 500 con una variabilidad alta (con esto nos referimos a que un tablero podía llegar a ser de 10×3 , no necesariamente siempre cuadrado).

Tal y como se puede observar en el segundo gráfico, la menor cota fue de 0.17 para un tablero de 263×165 con barcos [154, 152, 151, 142, 128, 99, 95, 90, 88, 71]. Esta es una cota muy baja, pero debemos observar el promedio de cotas para confirmar que este sea un caso aislado y no usual. Calculado la cota promedio nos da un valor de 0.942. Una cota promedio altísima, muy buena para un algoritmo de aproximación

Por otro lado, en el primer gráfico, se puede observar el tiempo de ejecución de el algoritmo de aproximación (prácticamente lineal) comparado con el de backtracking que llega a tardar mas de 500 segundos (8 minutos).

Tomando estos datos en cuenta podríamos concluir que es un algoritmo de aproximación bastante bueno. con una fiabilidad y grado de aproximación alto.

Finalmente, vamos a calcular la demanda cumplida para un tablero muy grande (inalcanzable para una resolución exacta de backtracking). El mecanismo que usaremos para verificar el grado de correctitud es la demanda cumplida sobre la demanda total ($\frac{\text{demanda cumplida}}{\text{demanda total}}$), ya que no tenemos forma de acceder a la cota para un caso como este por la imposibilidad de calcular con el algoritmo exacto.

En las iteraciones anteriores (las mostradas en el gráfico) la demanda cumplida sobre total promedio nos dio 0.14 (muy similar a la obtenida por BT, que fue 0.16). Ahora calculando tableros de 10.000 filas obtuvimos una demanda cumplida de 0.18. Por lo que aun con parametros muy altos se logro un comportamiento similar al registrado anteriormente.

6. Conclusiones

Demostramos que el problema de la batalla de naval está en NP y que es NP-Completo. Además mostramos dos formas de resolver el problema; mediante backtracking y el algoritmo de aproximación de John Jellicoe. Con este último método se consiguen tiempos sustancialmente mejores y una precisión bastante buena. Dependerá del problema a resolver y de la precisión de se necesite que algoritmo usar.